CHAPTER-3
PROCESS MANAGEMENT AND SYNCHRONIZATION
&
INTERPROCESS COMMUNICATION MECHANISMS

- Course Instructor: Dr. Venu Nalla

CPU SYNCRHONIZATION



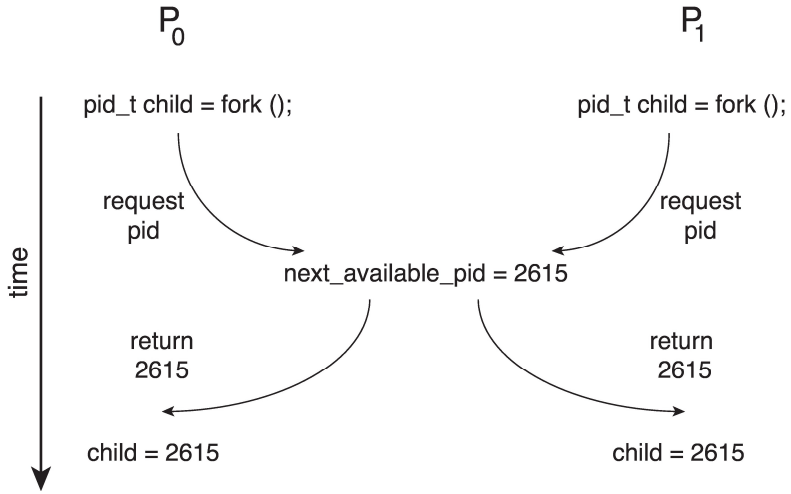
BACKGROUND

- Processes can execute concurrently
  - May be interrupted at any time, partially completing execution
- Concurrent access to shared data may result in data inconsistency
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes
- When we considered the Bounded Buffer problem with use of a counter that is updated concurrently by the producer and consumer,. it leads to race condition.

RACE CONDITION

- Processes P_0 and P_1 are creating child processes using the `fork()` system call

- Race condition on kernel variable `next_available_pid` which represents the next available process identifier (pid)



- Unless there is a mechanism to prevent P_0 and P_1 from accessing the variable `next_available_pid` the same pid could be assigned to two different processes!

CRITICAL SECTION PROBLEM

- Consider system of ***n*** processes $\{p_0, p_1, \ldots p_{n-1}\}$

- Each process has **critical section** segment of code
  - Process may be changing common variables, updating table, writing file, etc.
  - When one process in critical section, no other may be in its critical section

- ***Critical section problem*** is to design protocol to solve this

- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**

CRITICAL SECTION

- General structure of process P_i

```
while (true) {
        entry section
            critical section
        exit section
            remainder section
}
```

CRITICAL-SECTION PROBLEM (CONT.)

Requirements for solution to critical-section problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections

2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the process that will enter the critical section next cannot be postponed indefinitely

3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
   - Assume that each process executes at a nonzero speed
   - No assumption concerning **relative speed** of the ***n*** processes

INTERRUPT-BASED SOLUTION

- Entry section: disable interrupts
- Exit section: enable interrupts
- Will this solve the problem?
  - What if the critical section is code that runs for an hour?
  - Can some processes starve – never enter their critical section.
  - What if there are two CPUs?

SOFTWARE SOLUTION 1

- Two process solution
- Assume that the **`load`** and **`store`** machine-language instructions are atomic; that is, cannot be interrupted
- The two processes share one variable:
  - **`int turn;`**
- The variable **`turn`** indicates whose turn it is to enter the critical section
- initially, the value of **`turn`** is set to *i*

ALGORITHM FOR PROCESS P_I

```
while (true){

   while (turn = = j);

   /* critical section */

   turn = j;

   /* remainder section */

}
```

CORRECTNESS OF THE SOFTWARE SOLUTION

- Mutual exclusion is preserved

  P_i enters critical section only if:

  `turn = i`

  and `turn` cannot be both 0 and 1 at the same time

- What about the Progress requirement?

- What about the Bounded-waiting requirement?

PETERSON'S SOLUTION

- Two process solution
- Assume that the **`load`** and **`store`** machine-language instructions are atomic; that is, cannot be interrupted
- The two processes share two variables:
  - `int turn;`
  - `boolean flag[2]`
- The variable **`turn`** indicates whose turn it is to enter the critical section
- The **`flag`** array is used to indicate if a process is ready to enter the critical section.
  - `flag[i] = true` implies that process P_i is ready!

ALGORITHM FOR PROCESS P_I

```
while (true){

    flag[i] = true;
    turn = j;
    while (flag[j] && turn = = j)
        ;

        /* critical section */

    flag[i] = false;

     /* remainder section */

}
```

CORRECTNESS OF PETERSON'S SOLUTION

- Provable that the three CS requirement are met:
  1. Mutual exclusion is preserved

     P_i enters CS only if:

     either **`flag[j] = false`** or **`turn = i`**
  2. Progress requirement is satisfied
  3. Bounded-waiting requirement is met

PETERSON'S SOLUTION AND MODERN ARCHITECTURE

- Although useful for demonstrating an algorithm, Peterson's Solution is not guaranteed to work on modern architectures.
  - To improve performance, processors and/or compilers may reorder operations that have no dependencies
- Understanding why it will not work is useful for better understanding race conditions.
- For single-threaded this is ok as the result will always be the same.
- For multithreaded the reordering may produce inconsistent or unexpected results!

MODERN ARCHITECTURE EXAMPLE

- Two threads share the data:

```
boolean flag = false;
int x = 0;
```

- Thread 1 performs

```
while (!flag)
;
print x
```

- Thread 2 performs

```
x = 100;
flag = true
```

- What is the expected output?

100

MODERN ARCHITECTURE EXAMPLE (CONT.)

- However, since the variables `flag` and `x` are independent of each other, the instructions:

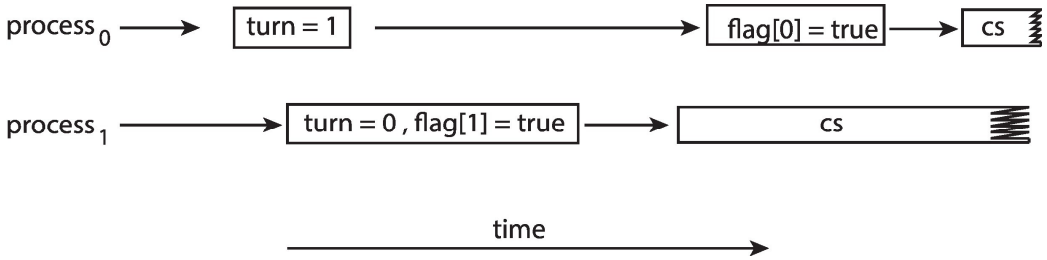
```
flag = true;
x = 100;
```

for Thread 2 may be reordered

- If this occurs, the output may be 0!

PETERSON'S SOLUTION REVISITED

- The effects of instruction reordering in Peterson's Solution



- This allows both processes to be in their critical section at the same time!

- To ensure that Peterson's solution will work correctly on modern computer architecture we must use **Memory Barrier**.

MEMORY BARRIER

- **Memory model** are the memory guarantees a computer architecture makes to application programs.

- Memory models may be either:
  - **Strongly ordered** – where a memory modification of one processor is immediately visible to all other processors.
  - **Weakly ordered** – where a memory modification of one processor may not be immediately visible to all other processors.

- A **memory barrier** is an instruction that forces any change in memory to be propagated (made visible) to all other processors.

MEMORY BARRIER INSTRUCTIONS

- When a memory barrier instruction is performed, the system ensures that all loads and stores are completed before any subsequent load or store operations are performed.

- Therefore, even if instructions were reordered, the memory barrier ensures that the store operations are completed in memory and visible to other processors before future load or store operations are performed.

MEMORY BARRIER EXAMPLE

- Returning to the previous example
- We could add a memory barrier to the following instructions to ensure Thread 1 outputs 100:
- Thread 1 now performs

```
   while (!flag)
 memory_barrier();
  print x
```

- Thread 2 now performs

```
   x = 100;
  memory_barrier();
  flag = true
```

- For Thread 1 we are guaranteed that that the value of `flag` is loaded before the value of `x`.
- For Thread 2 we ensure that the assignment to `x` occurs before the assignment `flag`.

SYNCHRONIZATION HARDWARE

- Many systems provide hardware support for implementing the critical section code.

- Uniprocessors – could disable interrupts
  - Currently running code would execute without preemption
  - Generally too inefficient on multiprocessor systems
    - Operating systems using this not broadly scalable

HARDWARE INSTRUCTIONS

- Special hardware instructions that allow us to either *test-and-modify* the content of a word, or to *swap* the contents of two words atomically (uninterruptedly.)
  - **Test-and-Set** instruction
  - **Compare-and-Swap** instruction

THE TEST_AND_SET INSTRUCTION

- Definition

```
boolean test_and_set (boolean *target)
  {
        boolean rv = *target;
        *target = true;
        return rv:
  }
```

- Properties
  - Executed atomically
  - Returns the original value of passed parameter
  - Set the new value of passed parameter to **`true`**

SOLUTION USING TEST_AND_SET()

- Shared boolean variable **`lock`**, initialized to **`false`**
- Solution:

```
do {
        while (test_and_set(&lock))
         ; /* do nothing */

             /* critical section */

     lock = false;
             /* remainder section */
 } while (true);
```

- Does it solve the critical-section problem?

THE COMPARE_AND_SWAP INSTRUCTION

- Definition

```
int compare_and_swap(int *value, int expected, int new_value)
 {
   int temp = *value;
   if (*value == expected)
       *value = new_value;
    return temp;
 }
```

- Properties
  - Executed atomically
  - Returns the original value of passed parameter **`value`**
  - Set the variable **`value`** the value of the passed parameter **`new_value`** but only if **`*value == expected`** is true. That is, the swap takes place only under this condition.

SOLUTION USING COMPARE_AND_SWAP

- Shared integer **`lock`** initialized to 0;
- Solution:

```
while (true){
    while (compare_and_swap(&lock, 0, 1) != 0)
        ; /* do nothing */

    /* critical section */

    lock = 0;

   /* remainder section */
}
```

- Does it solve the critical-section problem?

BOUNDED-WAITING WITH COMPARE-AND-SWAP

```
while (true) {
   waiting[i] = true;
   key = 1;
   while (waiting[i] && key == 1)
      key = compare_and_swap(&lock,0,1);
   waiting[i] = false;
   /* critical section */
   j = (i + 1) % n;
   while ((j != i) && !waiting[j])
      j = (j + 1) % n;
   if (j == i)
      lock = 0;
   else
      waiting[j] = false;
   /* remainder section */
}
```

ATOMIC VARIABLES

- Typically, instructions such as compare-and-swap are used as building blocks for other synchronization tools.
- One tool is an **atomic variable** that provides *atomic* (uninterruptible) updates on basic data types such as integers and booleans.
- For example:
  - Let **`sequence`** be an atomic variable
  - Let **`increment()`** be operation on the atomic variable **`sequence`**
  - The Command:

```
increment(&sequence);
```

ensures **`sequence`** is incremented without interruption:

ATOMIC VARIABLES

- The **`increment()`** function can be implemented as follows:

```
void increment(atomic_int *v)
{
 int temp;
 do {
     temp = *v;
 }
 while (temp != (compare_and_swap(v,temp,temp+1));
}
```

MUTEX LOCKS

- Previous solutions are complicated and generally inaccessible to application programmers
- OS designers build software tools to solve critical section problem
- Simplest is mutex lock
  - Boolean variable indicating if lock is available or not
- Protect a critical section by
  - First `acquire()` a lock
  - Then `release()` the lock
- Calls to `acquire()` and `release()` must be **atomic**
  - Usually implemented via hardware atomic instructions such as compare-and-swap.
- But this solution requires **busy waiting**
  - This lock therefore called a **spinlock**

SOLUTION TO CS PROBLEM USING MUTEX LOCKS

```
while (true) {
   acquire lock

        critical section

   release lock

remainder section
}
```

SEMAPHORE

- Synchronization tool that provides more sophisticated ways (than Mutex locks) for processes to synchronize their activities.
- Semaphore ***S*** – integer variable
- Can only be accessed via two indivisible (atomic) operations
  - **`wait()`** and **`signal()`**
    - Originally called **`P()`** and **`V()`**
- Definition of the **`wait()` operation**

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
```

- Definition of the **`signal()` operation**

```
signal(S) {
    S++;
}
```

SEMAPHORE (CONT.)

- **Counting semaphore** – integer value can range over an unrestricted domain
- **Binary semaphore** – integer value can range only between 0 and 1
  - Same as a **mutex lock**
- Can implement a counting semaphore ***S*** as a binary semaphore
- With semaphores we can solve various synchronization problems

SEMAPHORE USAGE EXAMPLE

- Solution to the CS Problem
  - Create a semaphore "**mutex**" initialized to 1

```
wait(mutex);
    CS
signal(mutex);
```

- Consider P_1 and P_2 that with two statements S_1 and S_2 and the requirement that S_1 to happen before S_2
  - Create a semaphore "**synch**" initialized to 0

```
P1:
    S1;
    signal(synch);
P2:
    wait(synch);
    S2;
```

SEMAPHORE IMPLEMENTATION

- Must guarantee that no two processes can execute the **`wait()`** and **`signal()`** on the same semaphore at the same time
- Thus, the implementation becomes the critical section problem where the **`wait`** and **`signal`** code are placed in the critical section
- Could now have **busy waiting** in critical section implementation
  - But implementation code is short
  - Little busy waiting if critical section rarely occupied
- Note that applications may spend lots of time in critical sections and therefore this is not a good solution

SEMAPHORE IMPLEMENTATION WITH NO BUSY WAITING

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
  - Value (of type integer)
  - Pointer to next record in the list
- Two operations:
  - **block** – place the process invoking the operation on the appropriate waiting queue
  - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue

IMPLEMENTATION WITH NO BUSY WAITING (CONT.)

- Waiting queue

```
typedef struct {
int value;
struct process *list;
} semaphore;
```

IMPLEMENTATION WITH NO BUSY WAITING (CONT.)

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

CLASSICAL PROBLEMS OF SYNCHRONIZATION

- Classical problems used to test newly-proposed synchronization schemes
  - Bounded-Buffer Problem
  - Readers and Writers Problem
  - Dining-Philosophers Problem

BOUNDED-BUFFER PROBLEM

- ***n*** buffers, each can hold one item
- Semaphore **`mutex`** initialized to the value 1
- Semaphore **`full`** initialized to the value 0
- Semaphore **`empty`** initialized to the value n

BOUNDED BUFFER PROBLEM (CONT.)

- The structure of the producer process

```
while (true) {
      ...
      /* produce an item in next_produced */
      ...
   wait(empty);
   wait(mutex);
      ...
      /* add next produced to the buffer */
      ...
   signal(mutex);
   signal(full);
}
```

BOUNDED BUFFER PROBLEM (CONT.)

- The structure of the consumer process

```
while (true) {
    wait(full);
    wait(mutex);
        ...
     /* remove an item from buffer to next_consumed */
        ...
    signal(mutex);
    signal(empty);
        ...
       /* consume the item in next consumed */
        ...
   }
```

READERS-WRITERS PROBLEM

- A data set is shared among a number of concurrent processes
  - **Readers** – only read the data set; they do ***not*** perform any updates
  - **Writers** – can both read and write
- Problem – allow multiple readers to read at the same time
  - Only one single writer can access the shared data at the same time
- Several variations of how readers and writers are considered – all involve some form of priorities

READERS-WRITERS PROBLEM (CONT.)

- Shared Data
  - Data set
  - Semaphore **`rw_mutex`** initialized to 1
  - Semaphore **`mutex`** initialized to 1
  - Integer **`read_count`** initialized to 0

READERS-WRITERS PROBLEM (CONT.)

- The structure of a writer process

```
while (true) {
      wait(rw_mutex);

         ...
      /* writing is performed */

         ...

   signal(rw_mutex);
}
```

READERS-WRITERS PROBLEM (CONT.)

- The structure of a reader process

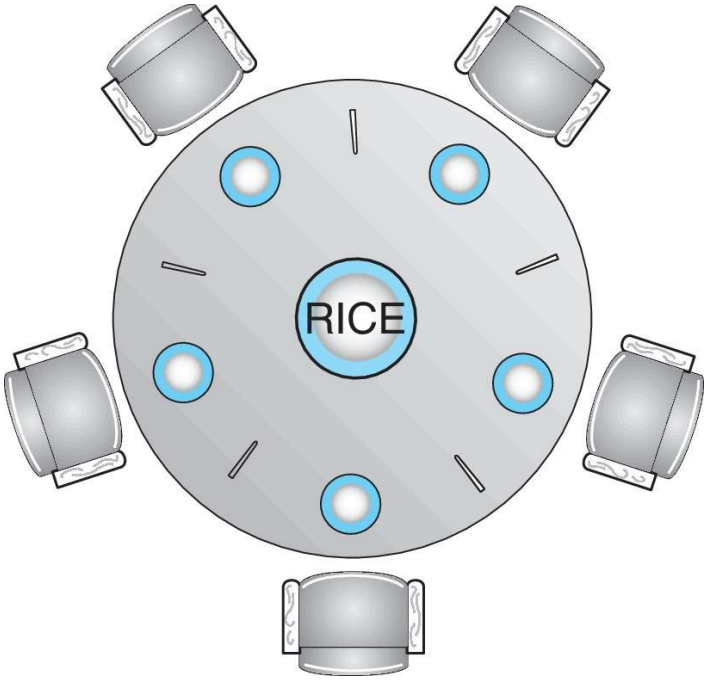
```
while (true){
     wait(mutex);
     read_count++;
     if (read_count == 1) /* first reader */
      wait(rw_mutex);
          signal(mutex);
        ...
        /* reading is performed */
        ...
     wait(mutex);
         read_count--;
         if (read_count == 0) /* last reader */
         signal(rw_mutex);
     signal(mutex);
}
```

READERS-WRITERS PROBLEM VARIATIONS

- The solution in previous slide can result in a situation where a writer process never writes. It is referred to as the "First reader-writer" problem.
- The "Second reader-writer" problem is a variation the first reader-writer problem that state:
  - Once a writer is ready to write, no "newly arrived reader" is allowed to read.
- Both the first and second may result in starvation. leading to even more variations
- Problem is solved on some systems by kernel providing reader-writer locks

DINING-PHILOSOPHERS PROBLEM

- N philosophers' sit at a round table with a bowel of rice in the middle.
- They spend their lives alternating thinking and eating.
- They do not interact with their neighbors.
- Occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
  - Need both to eat, then release both when done
- In the case of 5 philosophers, the shared data
  - Bowl of rice (data set)
  - Semaphore chopstick [5] initialized to 1



DINING-PHILOSOPHERS PROBLEM ALGORITHM

- Semaphore Solution
- The structure of Philosopher i :

```
while (true){
    wait (chopstick[i] );
    wait (chopStick[ (i + 1) % 5] );

     /* eat for awhile */

    signal (chopstick[i] );
    signal (chopstick[ (i + 1) % 5] );

     /* think for awhile */

}
```

- What is the problem with this algorithm?

DINING-PHILOSOPHERS PROBLEM ALGORITHM

- Suppose that all five philosophers become hungry at the same time and each grabs her left chopstick. All the elements of chopstick will now be equal to 0. When each philosopher tries to grab their right chopstick, they will be delayed forever.

- Several possible remedies to the deadlock problem are replaced by:
  - Allow at most four philosophers to be sitting simultaneously at the table.
  - Allow a philosopher to pick up her chopsticks only if both chopsticks are available (to do this, she must pick them up in a critical section).
  - Use an asymmetric solution—that is, an odd-numbered philosopher picks up first her left chopstick and then her right chopstick, whereas an even numbered philosopher picks up her right chopstick and then her left chopstick.

MONITORS

- A high-level abstraction that provides a convenient and effective mechanism for process synchronization
- *Abstract data type*, internal variables only accessible by code within the procedure
- Only one process may be active within the monitor at a time
- Pseudocode syntax of a monitor:

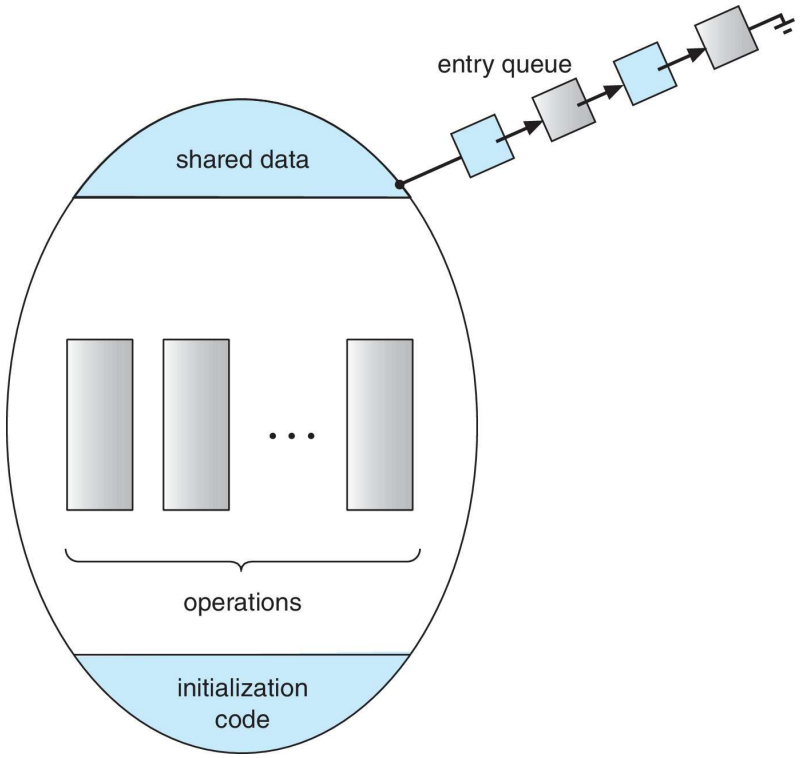
```
monitor monitor-name
{
  // shared variable declarations
  procedure P1 (…) { …. }

  procedure P2 (…) { …. }

  procedure Pn (…) {……}

  initialization code (…) { … }
}
```

SCHEMATIC VIEW OF A MONITOR



MONITOR IMPLEMENTATION USING SEMAPHORES

- Variables

```
semaphore mutex
mutex = 1
```

- Each procedure ***P*** is replaced by

```
wait(mutex);
    ...
    body of P;
    ...
signal(mutex);
```

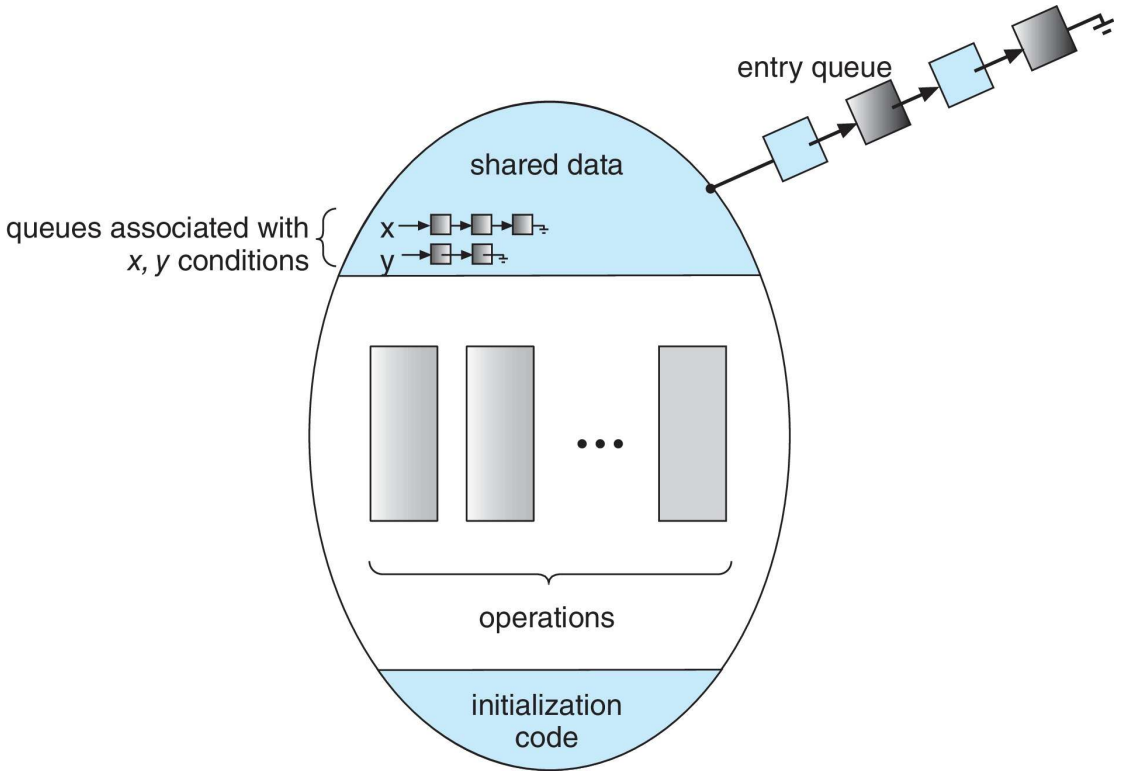
- Mutual exclusion within a monitor is ensured

CONDITION VARIABLES

- `condition x, y;`

- Two operations are allowed on a condition variable:
  - **`x.wait()`** – a process that invokes the operation is suspended until **`x.signal()`**
  - **`x.signal()`** – resumes one of processes (if any) that invoked **`x.wait()`**
    - If no **`x.wait()`** on the variable, then it has no effect on the variable

MONITOR WITH CONDITION VARIABLES



USAGE OF CONDITION VARIABLE EXAMPLE

- Consider P_1 and P_2 that that need to execute two statements S_1 and S_2 and the requirement that S_1 to happen before S_2
  - Create a monitor with two procedures F_1 and F_2 that are invoked by P_1 and P_2 respectively
  - One condition variable "x" initialized to 0
  - One Boolean variable "done"
  - **F1:**

```
S1;
done = true;
x.signal();
```

  - **F2:**

```
if done = false
   x.wait()
S2;
```

MONITOR IMPLEMENTATION USING SEMAPHORES

- Variables

```
semaphore mutex;  // (initially  = 1)
semaphore next;   // (initially  = 0)
int next_count = 0; // number of processes waiting
                        inside the monitor
```

- Each function ***P*** will be replaced by

```
wait(mutex);
      …
  body of P;
      …
if (next_count > 0)
  signal(next)
else
  signal(mutex);
```

- Mutual exclusion within a monitor is ensured

IMPLEMENTATION – CONDITION VARIABLES

- For each condition variable ***x***, we have:

```
semaphore x_sem; // (initially  = 0)
int x_count = 0;
```

- The operation **`x.wait()`** can be implemented as:

```
x_count++;
if (next_count > 0)
    signal(next);
else
   signal(mutex);
wait(x_sem);
x_count--;
```

IMPLEMENTATION (CONT.)

- The operation **`x.signal()`** can be implemented as:

```
if (x_count > 0) {
   next_count++;
   signal(x_sem);
   wait(next);
   next_count--;
}
```

MONITOR SOLUTION TO DINING PHILOSOPHERS

```
monitor DiningPhilosophers
{
   enum {THINKING; HUNGRY, EATING} state [5];
   condition self [5];

   void pickup (int i) {
          state[i] = HUNGRY;
          test(i);
          if (state[i] != EATING) self[i].wait;
   }

    void putdown (int i) {
          state[i] = THINKING;
                    // test left and right neighbors
          test((i + 4) % 5);
          test((i + 1) % 5);
   }
```

SOLUTION TO DINING PHILOSOPHERS (CONT.)

```
void test (int i) {
        if ((state[(i + 4) % 5] != EATING) &&
        (state[i] == HUNGRY) &&
        (state[(i + 1) % 5] != EATING) ) {
             state[i] = EATING ;
         self[i].signal () ;
        }
}

    initialization_code() {
      for (int i = 0; i < 5; i++)
      state[i] = THINKING;
     }
```

SOLUTION TO DINING PHILOSOPHERS (CONT.)

- Each philosopher "i" invokes the operations **`pickup()`** and **`putdown()`** in the following sequence:

```
DiningPhilosophers.pickup(i);

    /** EAT **/

DiningPhilosophers.putdown(i);
```

- No deadlock, but starvation is possible

RESUMING PROCESSES WITHIN A MONITOR

- If several processes queued on condition variable **`x`**, and **`x.signal()`** is executed, which process should be resumed?

- FCFS frequently not adequate

- Use the **conditional-wait** construct of the form

  `x.wait(c)`

  where:

  - **`c`** is an integer (called the priority number)
  - The process with lowest number (highest priority) is scheduled next

Single Resource allocation

- Allocate a single resource among competing processes using priority numbers that specifies the maximum time a process plans to use the resource

```
R.acquire(t);
    ...
  access the resurce;
    ...

R.release;
```

- Where R is an instance of type **ResourceAllocator**

Single Resource allocation

- Allocate a single resource among competing processes using priority numbers that specifies the maximum time a process plans to use the resource
- The process with the shortest time is allocated the resource first
- Let R is an instance of type **`ResourceAllocator`** (next slide)
- Access to **`ResourceAllocator`** is done via:

```
R.acquire(t);
      ...
  access the resurce;
      ...
R.release;
```

- Where **`t`** is the maximum time a process plans to use the resource

A MONITOR TO ALLOCATE SINGLE RESOURCE

```
monitor ResourceAllocator
{
   boolean busy;
   condition x;
   void acquire(int time) {
            if (busy)
               x.wait(time);
            busy = true;
   }
   void release() {
            busy = false;
            x.signal();
   }
   initialization code() {
    busy = false;
   }
}
```

SINGLE RESOURCE MONITOR (CONT.)

- Usage:

```
acquire
  ...
release
```

- Incorrect use of monitor operations
  - `release()` … `acquire()`
  - `acquire()` … `acquire())`
  - Omitting of `acquire()` and/or `release()`

LIVENESS

- Processes may have to wait indefinitely while trying to acquire a synchronization tool such as a mutex lock or semaphore.
- Waiting indefinitely violates the progress and bounded-waiting criteria discussed at the beginning of this chapter.
- **Liveness** refers to a set of properties that a system must satisfy to ensure processes make progress.
- Indefinite waiting is an example of a liveness failure.

LIVENESS

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let ***S*** and ***Q*** be two semaphores initialized to 1

P_0	P_1
`wait(S);`	`wait(Q);`
`wait(Q);`	`wait(S);`
`...`	`...`
`signal(S);`	`signal(Q);`
`signal(Q);`	`signal(S);`

- Consider if P_0 executes wait(S) and P_1 wait(Q). When P_0 executes wait(Q), it must wait until P_1 executes signal(Q)
- However, P_1 is waiting until P_0 execute signal(S).
- Since these signal() operations will never be executed, P_0 and P_1 are **deadlocked**.

LIVENESS

- Other forms of deadlock:

- **Starvation** – indefinite blocking
  - A process may never be removed from the semaphore queue in which it is suspended

- **Priority Inversion** – Scheduling problem when lower-priority process holds a lock needed by higher-priority process
  - Solved via **priority-inheritance protocol**